Software Design Patterns

Assignment 3: Bridge Pattern

Instructor: Bekenov Yerassyl

Amerkhanova Aruzhan
Group: SE-2529
Topic: A — Drawing

Submission date: 04.10.2026

Repository: https://github.com/xelllion/SDP_assignment3.git

Base commit: b5ba7939358465442970081945249204b5f633c5

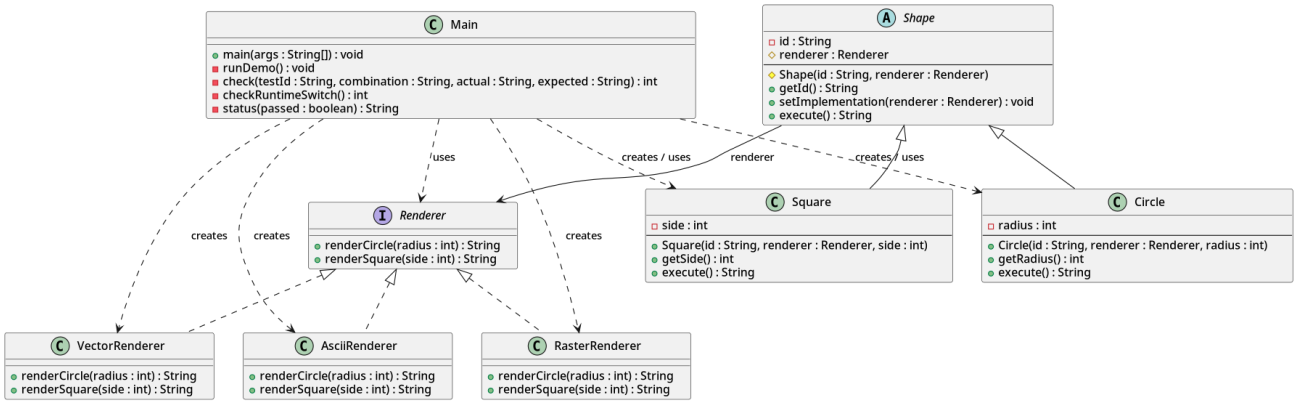
Submitted commit: In Moodle submission text

1. Bridge Pattern Overview

The application demonstrates the Bridge design pattern using two independently varying hierarchies. The abstraction hierarchy consists of Shape, Circle, and Square. The implementation hierarchy consists of Renderer, VectorRenderer, RasterRenderer, and AsciiRenderer.

Shape stores a reference of type Renderer. Therefore, a shape does not depend on a concrete rendering implementation. Circle and Square delegate rendering work to the current Renderer object. Bridge is suitable because shapes and rendering methods can evolve independently without creating a separate subclass for every Shape/Renderer combination.

2. UML Class Diagram



The diagram shows inheritance on the abstraction side, interface realization on the implementation side, the Renderer-typed bridge reference stored by Shape, and the client dependencies from Main.

3. Clean Code Examples

3.1 Separation of responsibilities

```
public interface Renderer {
    String renderCircle(int radius);
    String renderSquare(int side);
}
```

The Renderer interface separates low-level rendering behavior from the Shape hierarchy. Shape classes keep shape-specific state, while renderer classes are responsible for rendering.

3.2 Meaningful names

```
public void setImplementation(Renderer renderer) {
    this.renderer = renderer;
}
```

Names such as renderer, radius, side, and setImplementation clearly communicate the purpose of fields, parameters, and methods.

3.3 Small focused methods

```
@Override
public String execute() {
    return renderer.renderCircle(radius);
}
```

The execute method has one focused responsibility: delegate Circle rendering to the current Renderer.

3.4 Avoidance of duplication

```
private static int check(
        String testId,
        String combination,
        String actual,
        String expected) {
```

The common check method is reused by T1, T2, T3, T4, T6, and T7. This avoids duplicating result comparison and PASS/FAIL output logic.

3.5 Encapsulation of state

```
private final int radius;

public int getRadius() {
    return radius;
}
```

The Circle radius is private and final, so it cannot be modified directly from outside the class. Its value is exposed only through a getter.

4. Runtime Switching (T5)

T5 creates one Circle object using VectorRenderer, stores a reference to the original object, executes it, and then replaces only the implementation with RasterRenderer through setImplementation(...). The test uses == to prove that the same Circle object is still used. It also checks that the Circle ID and radius remain unchanged while the rendering result changes from vector to raster.

5. Independent Extension with I3

The initial working solution contained VectorRenderer and RasterRenderer and demonstrated T1-T5. After that version was committed as the base commit, AsciiRenderer was added as the third implementation.

AsciiRenderer implements the existing Renderer interface. No changes were required in Shape, Circle, Square, Renderer, VectorRenderer, or RasterRenderer. Inside src/, only the new AsciiRenderer class and Main were changed for the extension. This demonstrates that the implementation hierarchy can be extended independently from the abstraction hierarchy.

6. Bridge Trade-off

One trade-off of Bridge is that it introduces more classes and interfaces. For a very small application, this may make the design look more complex. The extra structure is useful when both dimensions are expected to vary independently.

7. Bridge vs Adapter

Bridge and Adapter may both use composition, but their intentions are different. Bridge is designed to separate an abstraction from its implementation so both can vary independently. Adapter is used to make an existing incompatible interface work with the interface expected by a client. In this project, Renderer is an intentional implementation hierarchy rather than a wrapper around a legacy or incompatible interface, so Bridge is the appropriate pattern.

8. Demonstration Checks

Test	Action	Expected result
T1	Circle + VectorRenderer	VECTOR circle radius=2
T2	Circle + RasterRenderer	RASTER circle radius=2
T3	Square + VectorRenderer	VECTOR square side=3
T4	Square + RasterRenderer	RASTER square side=3
T5	Runtime switch Vector -> Raster	Same object; unchanged ID and radius
T6	Circle + AsciiRenderer	ASCII circle radius=2
T7	Square + AsciiRenderer	ASCII square side=3

9. References

1. Lecture 4 — Bridge Pattern, Software Design Patterns course.

2. Assignment 3 — Bridge Pattern instructions.

3. Bridge Pattern reading provided in Moodle.